

Lecture 4: Threads

Emergency Exam Walkthrough and Mastery Guide

Operating Systems - CS x61 | Dr. Noha Adly

Read this first: the entire lecture in one sentence

- A process owns resources, while a thread is the schedulable execution path inside that process.
- Threads share the process address space and resources, but each thread has its own PC, registers, stack, and state.
- The main exam fight is ULT vs KLT vs combined approaches, plus states, Pthreads, Amdahl law, and case studies.

0. Two-Hour Emergency Study Plan

Time	Study target	What you must be able to say
0-15 min	Core idea	Process = resource container. Thread = execution unit. Know shared vs private items.
15-35 min	Thread states and operations	Running, Ready, Blocked. Spawn, Block, Unblock, Finish. Process termination kills all threads.
35-60 min	ULT vs KLT vs combined	Management location, kernel awareness, blocking behavior, multiprocessing, overhead.
60-75 min	Pthreads + performance	pthread_create, exit, join, yield, attr calls. Null fork and signal-wait comparison.
75-95 min	Amdahl law	Serial portion limits speedup. Formula and examples.
95-120 min	Case studies	Windows, Solaris, Linux, Android, GCD. Focus on special terms and states.

1. Master Cheat Sheet

Concept	Exam answer
Process	Infrastructure/resource container where execution happens: address space + resources.
Thread	A program/execution path inside a process context. Also called a lightweight process in some contexts.
Resource ownership	Belongs mainly to the process: address space, I/O channels, devices, files.
Scheduling/execution	Belongs mainly to the thread: it has a PC, registers, stack, state, and can be dispatched.
Thread Control Block (TCB)	Stores thread-specific context such as register values, priority, state, etc.
Shared by threads	Code, data, address space, open files, signals/signal handlers, global variables, process resources.
Private per thread	Thread ID, program counter, register set, stack, execution state.
Global process actions	Suspending/swapping a process affects all threads. Terminating a process terminates all its threads.
ULT	User-Level Thread: managed by application/library; kernel does not know individual threads.
KLT	Kernel-Level Thread: managed and scheduled by kernel; can run on multiple processors.
Combined approach	Maps user threads to kernel threads/LWPs to combine ULT flexibility with KLT concurrency.
Amdahl law	Speedup is limited by the serial part of the program, even if many cores are added.

Memory line to repeat before the exam

- Process owns resources. Thread owns execution state.
- Same process threads share memory and files, but each thread has its own stack and registers.
- ULT is fast but kernel-blind. KLT is kernel-aware but costs kernel mode switching.

- More cores help only the parallel part; serial code limits maximum speedup.

2. Processes and Threads: The Core Split

The lecture starts by separating two things that are mixed together in a normal process: resource ownership and execution/scheduling.

Process characteristic	Meaning	When separated, it becomes
Resource ownership	Address space, I/O channels, devices, files, resources.	Process or task
Scheduling / execution	Execution path, CPU dispatching, states like Running and Ready.	Thread or lightweight process

A process is the infrastructure where execution takes place. It provides the address space and resources. A thread is the execution path inside that process.

A traditional heavyweight process has one thread of control. A multithreaded process has multiple threads inside the same process.

Diagram logic you should understand

Think of one process as a house. The house contains shared rooms: code, data, files, and address space. Each thread is a person walking inside the house with its own backpack: stack, registers, program counter, and state.

Inside one process	Shared or private?	Why it matters
Code section	Shared	All threads can execute the same program code.
Data section / global variables	Shared	One thread changing shared data can affect the others.
Open files and signals	Shared	If one thread opens a readable file, other threads in the same process can read it.
Stack	Private per thread	Each thread needs its own function calls, local variables, and return addresses.
Program counter and registers	Private per thread	Each thread may be executing a different instruction.
Execution state	Private per thread	One thread can be Running, another Ready, another Blocked.

3. Classical Thread Model

The classical model divides information into per-process items and per-thread items. This table is very testable because it directly answers: what belongs to the process and what belongs to the thread?

Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	Thread ID / thread context
Signals and signal handlers	Priority / scheduling info
Accounting information	Kernel stack and user stack in many OS diagrams

Common mistake

- Do not say every thread has its own full address space. Threads in the same process share the same address space.
- Do not say all thread operations are global. Thread states are per thread, but process termination/suspension is global to all threads.

4. Single-Threaded vs Multithreaded Approaches

System / approach	Meaning
MS-DOS	Single user process and a single thread.
Some older UNIX variants	Multiple user processes, but one thread per process.
Java runtime environment	One process with multiple threads.

Windows, Solaris, modern UNIX/Linux, Mach, OS/2	Multiple processes, and each process may support multiple threads.
---	--

Thread usage examples

Example	Thread roles	Why threads help
Word processor	1) User interaction. 2) Background reformatting. 3) Periodic saving from RAM to disk.	The UI can stay responsive while formatting or saving happens.
Web server	Dispatcher thread receives/assigns work; worker threads serve requests; cache is shared inside the server process.	Multiple client requests can be handled concurrently without creating a whole new process for every request.
General modern app	Display update, fetch data, spell checking, network request handling.	Different tasks can progress at the same time or be interleaved.

5. Benefits of Threads

Benefit	Explanation	Exam phrase
Responsiveness	Part of a process can continue even if another thread is blocked.	Important for user interfaces.
Resource sharing	Threads share the parent process resources.	Easier than shared memory or message passing between processes.
Efficiency	Threads in one process can communicate through shared memory and files without invoking the kernel.	Less overhead.
Economy	Thread creation and termination are cheaper than process creation/termination.	Lecture mentions thread creation around 30x cheaper and thread switching around 5x lower overhead.
Scalability	Threads can take advantage of multiprocessor/multicore architectures.	Parallel threads may run on different cores.
Code simplicity	Separate tasks can be written as separate threads.	Cleaner structure for applications with concurrent tasks.

6. Thread Execution States and Operations

The key thread states in the lecture are Running, Ready, and Blocked. The same process can contain threads in different states.

State / operation	Meaning
Running	The thread is currently executing on the CPU.
Ready	The thread can run, but is waiting to be selected by the scheduler.
Blocked	The thread is waiting for an event, I/O, resource, or another condition.
Spawn	Create another thread with its own register context and stack.
Block	Thread waits for an event; user registers, PC, and stack are saved.
Unblock	The event happens; thread moves to the Ready queue.
Finish	Thread completes; register context and stacks are deallocated.

Important question: if one thread blocks, do all threads block?

- Generally, no. A thread can block without blocking the remaining threads in the process.
- This is the point of using threads: the process can start multiple operations, and each thread can block on its own event.
- However, in pure User-Level Threads, a blocking system call can block the whole process because the kernel only sees the process, not the individual ULTs.

Multithreading on a uniprocessor

On a single processor, threads do not literally run at the same instant. The OS interleaves them. Execution switches from one thread to another when the current thread blocks or its time quantum expires.

Event	What happens
I/O request	The running thread may become blocked.
Request complete	The blocked thread becomes ready again.
Time quantum expires	The scheduler may preempt the current thread and run another ready thread.

New process/thread created	It enters the scheduling system and may become ready/running later.
----------------------------	---

Threads and RPC

If a program performs two remote procedure calls, a single thread may do them one after another and wait during each call. With one thread per server/request, the waits can overlap, improving total completion time even on a uniprocessor because the CPU can switch while one thread is blocked waiting for a response.

7. POSIX Threads / Pthreads

POSIX means Portable Operating Systems based on UNIX. POSIX Threads, usually called Pthreads, is an IEEE API standard for thread operations.

Pthread call	Meaning
<code>pthread_create()</code>	Create a new thread.
<code>pthread_exit()</code>	Terminate the calling thread.
<code>pthread_join()</code>	Wait for a specific thread to exit.
<code>pthread_yield()</code>	Release the CPU so another thread can run.
<code>pthread_attr_init()</code>	Create and initialize a thread attribute structure.
<code>pthread_attr_destroy()</code>	Remove/destroy a thread attribute structure.
<code>pthread_mutex_lock()</code>	Acquire a mutex for synchronization.
<code>pthread_mutex_unlock()</code>	Release a mutex.
<code>pthread_mutex_trylock()</code>	Try to acquire a mutex without blocking like a normal lock.
<code>pthread_cond_wait()</code>	Wait on a condition variable.
<code>pthread_cond_signal()</code>	Signal a condition variable.

How to read the Pthreads code example

1. The program defines `NUMBER_OF_THREADS = 10`.
2. `main()` creates an array of `pthread_t` thread identifiers.
3. A loop calls `pthread_create(&threads[i], NULL, print_hello_world, (void *) i)`.
4. Each created thread runs `print_hello_world()`, prints its ID, then calls `pthread_exit(NULL)`.
5. If `pthread_create` returns a nonzero status, the code treats it as an error and exits.

8. How Threads Are Implemented

Approach	Who manages threads?	Kernel aware of each thread?	Main advantage	Main disadvantage
User-Level Threads (ULT)	User-level thread library / application.	No. Kernel schedules the process as one unit.	Very fast user-space switching; application-specific scheduling; works without kernel changes.	Blocking system call can block all threads in the process; cannot exploit multiprocessing by itself.
Kernel-Level Threads (KLT)	Kernel.	Yes. Kernel schedules threads individually.	Can schedule multiple threads of same process on multiple processors; one blocked thread does not block all; kernel routines can be multithreaded.	Switching between threads requires a mode switch to the kernel.
Combined approach	Thread creation/synchronization mostly in user space, but ULTs map to KLTs/LWPs.	Kernel sees kernel-level execution entities.	Combines ULT flexibility with KLT parallelism and nonblocking behavior.	More complex mapping and tuning.

User-Level Threads (ULT) details

- All thread management is done by the application or thread library.
- The kernel is not aware that multiple threads exist inside the process.
- The kernel gives the process a single state: Ready, Running, Blocked, etc.
- The thread library handles creating, destroying, scheduling, saving/restoring thread contexts, and passing messages/data between threads.
- Threads only execute on the processor when the whole process is in Running state.

ULT and process-state trap scenarios

Scenario	Kernel view	Thread-library view	Exam lesson
Thread 2 makes a blocking system call.	Kernel blocks process B and switches to another process.	Thread 2 may still be perceived as Running by the library.	In pure ULT, a blocking system call can block the entire process.

Process B exhausts its time slice.	Kernel moves B to Ready and switches to another process.	Thread 2 may still be marked Running inside the library.	A ULT can be library-running but not actually executing because the process is not on the CPU.
Thread 2 needs action by Thread 1.	Process B can remain Running.	Thread 2 becomes Blocked; Thread 1 moves Ready -> Running.	A user-level block inside the library does not necessarily block the whole process.

Kernel-Level Threads (KLT) details

- All thread management is done by the kernel; no thread-management code is needed inside the user program.
- The kernel maintains context information for the process and for each individual thread.
- Scheduling is done by the kernel on a thread basis.
- The kernel can run multiple threads of one process on multiple processors.
- If one thread blocks, the kernel can schedule another thread from the same process.
- Disadvantage: switching from one thread to another in the same process requires entering kernel mode.

Performance comparison: Process vs ULT vs KLT

The lecture compares operation latencies on a VAX running a UNIX-like OS. Memorize the direction: ULT is fastest, KLT is slower, full processes are slowest.

Operation	Meaning	ULT latency (microseconds)	KLT latency (microseconds)	Process latency (microseconds)
Null fork	Time to create, schedule, execute, and complete a process/thread that invokes a null procedure.	34	948	11,300
Signal-wait	Time for a process/thread to signal a waiting process/thread and then wait on a condition.	37	441	1,840

Combined approach details

Solaris is the example used in the lecture. Thread creation, much of scheduling, and synchronization are done in user space, while user threads are mapped onto an equal or smaller number of kernel-level threads.

- The programmer can adjust the number of KLTs for an application and processor to improve results.
- Multiple threads within the same application can run in parallel on multiple processors.
- If one thread is blocked, another thread in the same application does not necessarily block.
- This approach tries to combine the advantages of ULT and KLT.

Relationship between Threads and Processes

Careful: the lecture table is labeled Threads:Processes. Do not flip the ratio.

Threads:Processes	Description	Example systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	One process defines address space and resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another, allowing movement among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M.	TRIX

9. Amdahl Law: Multicore Speedup

Amdahl law identifies the performance gain from adding cores to an application that has a serial part and a parallel part.

Formula
- $\text{Speedup} = 1 / (S + ((1 - S) / N))$
- S = serial fraction of the application.
- N = number of processing cores.
- As N approaches infinity, maximum speedup approaches $1 / S$.

Example	Calculation idea	Result
75% parallel, 25% serial, 2 cores	$1 / (0.25 + 0.75/2)$	1.6x speedup

75% parallel, 25% serial, 4 cores	$1 / (0.25 + 0.75/4)$	2.28x speedup
40% serial, infinite cores	Maximum = $1 / 0.4$	2.5x maximum speedup

Exam interpretation: the serial portion has a disproportionate effect. Even infinite cores cannot speed up the serial part. Overheads also reduce real speedup, so practical results are often worse than the ideal curve.

10. Case Studies

Windows

Windows treats processes and threads as objects. A process owns resources such as memory and open files. A thread is the dispatchable unit of work: it executes sequentially and can be interrupted so the processor can run another thread.

- A process must contain at least one thread to execute.
- Threads in different processes can execute concurrently.
- Multiple threads in the same process may be allocated to separate processors.
- Threads in the same process exchange information through shared address space and shared resources.

Windows process and thread object attributes

Process object attributes	Thread object attributes
Process ID	Thread ID
Security descriptor	Thread context
Base priority	Dynamic priority
Default processor affinity	Base priority
Quota limits	Thread processor affinity
Execution time	Thread execution time
I/O counters	Alert status
VM operation counters	Suspension count
Exception/debugging ports	Impersonation token
Exit status	Termination port and thread exit status

Professor trap: some thread attributes resemble process attributes because the thread attribute value may be derived from the process attribute value.

Windows thread states

State	Meaning / transition
Ready	Runnable and waiting to be selected.
Standby	Selected/picked to run next.
Running	Currently executing.
Waiting	Blocked/suspended waiting for a resource or event.
Transition	Resource is not available yet; moves to Ready when resource becomes available.
Terminated	Thread has ended.
Preempted	Running -> Ready when another thread/time-slice event takes CPU.
Block/Suspend	Running -> Waiting.
Unblock/Resume + resource available	Waiting -> Ready.
Unblock + resource not available	Waiting -> Transition.

Windows SMP support and affinity

- Threads can run on any processor by default.
- The kernel assigns a Ready thread to the next available processor.
- Soft affinity: dispatcher tries to run a Ready thread on the same processor it ran on before, so cache data may be reused.
- Hard affinity: an application restricts a thread to certain processors.

Solaris

Solaris concept	Meaning
Process	Includes user address space, stack, and process control block.
User-Level Threads (ULTs)	Implemented through a thread library, invisible to the OS, user-created execution units inside a process.
Kernel threads	Fundamental entities that can be scheduled to run on processors.
Lightweight Processes (LWPs)	Mapping layer between ULTs and kernel threads. Each LWP supports ULTs and maps to exactly one kernel thread.

There is exactly one kernel thread for each LWP. A single ULT bound to a single LWP gives one thread of execution and no concurrency. If concurrency is needed, the process has multiple ULTs, each bound to LWPs, each bound to kernel threads.

Solaris state	Meaning
RUN	Ready to execute.
ONPROC	Executing on a processor.
SLEEP	Blocked / waiting for an event.
STOP	Stopped, for example during debugging.
ZOMBIE	Thread has terminated.
FREE	Thread resources released.
ONPROC -> RUN	If preempted by a higher-priority thread or time-slicing.
ONPROC -> SLEEP	If waiting for an event.
STOP entry	If the process is stopped, e.g., for debugging.

Linux

In Linux, a process is represented by a `task_struct` data structure.

task_struct category	Meaning
State	Execution state of the process.
Scheduling information	Normal or real-time process plus priority.
Identifiers	IDs used to identify the process.
Links	Links to parent, siblings, and children.
Times and timers	Creation time, processor time consumed, interval timers.
File system	Pointers to files opened by the process.
Address space	Virtual address space assigned to the process.
Processor-specific context	Registers and stack information that form the process context.

Linux state	Meaning
Running	Either executing or ready.
Interruptible	Blocked, waiting for an event/resource/signal; can handle signals.
Uninterruptible	Blocked directly on hardware conditions; will not handle signals.
Stopped	Halted; can resume through action from another process, e.g., debugging.
Zombie	Terminated but still has its PCB/process table entry.

Linux threads

- Older Linux had no direct kernel multithreading support; applications used user-level POSIX library functions, mapping all threads to one kernel-level process.
- Modern Linux uses a mechanism similar to Solaris LWP.
- User-level threads are mapped to kernel-level processes, and processes can share resources.
- `clone()` creates a new process-like execution unit by copying attributes of the current process and optionally sharing files, signals, and virtual memory.
- Linux has no separate basic type called thread in the same way as some systems; threads are task-like entities that share selected resources.

Android process and thread management

Android component	Meaning
Activities	Screens users interact with, e.g., show email list, compose email, read email.
Services	Background operations, e.g., play music or fetch data over the network without blocking user interaction.
Content providers	Interface to application data, e.g., NotePad saving notes.
Broadcast receivers	Respond to system-wide broadcasts, e.g., low battery warning.

Each Android application runs on its own dedicated virtual machine and its own single process. This is the sandboxing model, which isolates each application.

Mac OS X Grand Central Dispatch (GCD)

- Provides a pool of available threads.
- Programmers designate portions of applications called blocks that can be dispatched independently and run concurrently.
- Concurrency depends on available cores and system thread capacity.
- A block is a self-contained unit of work and can encapsulate complex functions.
- Blocks are scheduled and dispatched by queues on a first-in-first-out basis.
- A queue can be associated with an event source such as a timer, network socket, or file descriptor.

11. Professor-Trap Details

Trap	Correct answer
Thread vs process	Process owns resources; thread is the execution/dispatch unit.
Stack question	Each thread has its own stack. Threads do not share one stack.
Address space question	Threads inside the same process share the same address space.
File question	If one thread opens a file with read privileges, other threads in the same process can also read it.
Scheduling question	Scheduling and dispatching are done on a thread basis.
Global action question	Suspending/swapping or terminating a process affects all its threads.
ULT blocking question	A blocking system call in one ULT can block the whole process because the kernel sees only the process.
KLT overhead question	KLT switching needs kernel mode, so it has more overhead than ULT switching.
Multiprocessing question	Pure ULT cannot take advantage of multiprocessing, but KLT and combined approaches can.
Amdahl question	Adding cores does not speed up the serial part. Maximum speedup is limited by $1/S$.
Relationship table question	The ratio is Threads:Processes in the lecture table.
Windows affinity question	Soft affinity is a dispatcher preference for cache reuse; hard affinity is an application restriction.
Linux clone question	<code>clone()</code> creates a process-like entity that can share resources such as files, signals, and virtual memory.

12. Likely Exam Questions and Model Answers

Q: Define a thread.

A: A thread is the basic unit of CPU utilization/execution inside a process. It has its own thread ID, program counter, register set, stack, and state, while sharing the process code, data, files, and resources.

Q: What is the difference between a process and a thread?

A: A process is a resource-owning infrastructure with address space and resources. A thread is an execution path inside the process and is the unit scheduled/dispatchable by the CPU.

Q: What do threads in the same process share?

A: They share the address space, code, data/global variables, open files, signals/signal handlers, and other process resources.

Q: What is private to each thread?

A: Program counter, registers, stack, state, thread ID, and thread context/TCB information.

Q: Why are threads cheaper than processes?

A: Threads share process resources, so creating/switching/terminating a thread needs less resource allocation and less context change than a process.

Q: Give two thread usage examples.

A: Word processor: UI, formatting, autosave. Web server: dispatcher thread and worker threads handling requests.

Q: List thread operations.

A: Spawn, block, unblock, finish.

Q: Can one thread block without blocking the remaining threads?

A: Generally yes, but pure ULTs can block the whole process if the blocked operation is a blocking system call handled by the kernel.

Q: What is a User-Level Thread?

A: A thread managed by a user-level library. The kernel is unaware of individual threads and schedules the entire process as one unit.

Q: Advantages of ULT?

A: Fast switching without kernel mode, application-specific scheduling, works on OSs without kernel thread support.

Q: Disadvantages of ULT?

A: Blocking system call can block all process threads; cannot exploit multiprocessing by itself.

Q: What is a Kernel-Level Thread?

A: A thread managed and scheduled by the kernel, with kernel-maintained context for each thread.

Q: Advantages of KLT?

A: Kernel can schedule threads from the same process on multiple processors; one blocked thread does not block all; kernel routines can be multithreaded.

Q: Disadvantage of KLT?

A: Thread switching requires a mode switch to the kernel, adding overhead.

Q: What is the combined approach?

A: A hybrid where user threads are mapped to kernel threads/LWPs, combining user-level management with kernel-level scheduling and parallelism.

Q: State Amdahl law.

A: $\text{Speedup} = 1 / (S + ((1-S)/N))$, where S is the serial fraction and N is the number of cores. Maximum speedup approaches $1/S$.

Q: What is pthread_join()?

A: It waits for a specific thread to exit.

Q: What is pthread_yield()?

A: It releases the CPU to let another thread run.

Q: What are Solaris LWPs?

A: Lightweight processes that map user-level threads to kernel threads; each LWP maps to exactly one kernel thread.

Q: What is Android sandboxing?

A: Each Android application runs in its own dedicated virtual machine and its own process, isolating it from other applications.

13. Final Rapid Flashcards

Front	Back
Process = ?	Resource owner: address space + resources.
Thread = ?	Execution/dispatch unit inside a process.
TCB contains?	Registers, priority, state, thread context, etc.
Per-thread items?	PC, registers, stack, state.
Per-process items?	Address space, globals, files, signals, child processes, accounting.
Process termination causes?	All its threads terminate.
Process suspension causes?	All its threads are suspended/swapped.
ULT managed by?	User-level thread library/application.
KLT managed by?	Kernel.
ULT kernel awareness?	Kernel is unaware of individual threads.
KLT kernel awareness?	Kernel knows and schedules individual threads.
ULT big advantage?	Fast switching without kernel mode.
ULT big disadvantage?	Blocking system call can block whole process; no multiprocessing by itself.
KLT big advantage?	Can run same-process threads on multiple processors; blocked thread does not block all.
KLT big disadvantage?	Thread switch requires kernel mode.
Combined approach example?	Solaris.
Null fork measures?	Overhead of creating/scheduling/executing/completing process/thread with null procedure.
Signal-wait measures?	Overhead of synchronizing two processes/threads.
Amdahl S means?	Serial fraction.
Amdahl N means?	Number of cores.
Max speedup as N -> infinity?	$1/S$.
Windows soft affinity?	Prefer same processor to reuse cache.
Windows hard affinity?	Application restricts thread to certain processors.
Solaris RUN?	Ready to execute.
Solaris ONPROC?	Executing on processor.
Linux interruptible?	Blocked but can handle signals.
Linux uninterruptible?	Blocked on hardware; does not handle signals.
GCD block?	Self-contained unit of work scheduled by queues.

14. What to Say if Asked to Explain the Whole Lecture

This lecture explains why operating systems separate resource ownership from execution. A process mainly owns resources such as address space, files, devices, and other OS resources. A thread is the execution path inside the process and is the

unit that the CPU scheduler can dispatch. Threads in the same process share code, data, files, and address space, but every thread has its own program counter, registers, stack, and state.

Threads are useful because they improve responsiveness, share resources naturally, reduce overhead compared with processes, and can scale on multiprocessor systems. The lecture then compares user-level threads, kernel-level threads, and combined approaches. ULTs are fast because management happens in user space, but the kernel is unaware of them, so blocking system calls and multiprocessing are problems. KLTs are managed by the kernel, allowing real parallelism and better blocking behavior, but switching costs more because it requires kernel mode. Combined approaches, such as Solaris, map ULTs onto LWPs/kernel threads to get advantages of both.

The lecture also introduces Pthreads as a standard thread API, Amdahl law to explain why serial code limits speedup on multiple cores, and OS case studies showing how Windows, Solaris, Linux, Android, and Mac OS X manage processes and threads.